Reverse proxy Nginx avec Docker Compose

Documentation de la mise en place d'un reverse proxy Nginx devant deux applications de test, le tout géré par Docker Compose. Rédigée à partir de la session de terminal du 10/11/2026 sur le serveur `gabgab`.

Objectif

Exposer plusieurs applications conteneurisées derrière un seul point d'entrée (ports 80 et 443). Nginx reçoit les requêtes et les redirige vers le bon conteneur selon la configuration.

Architecture

Service	Conteneur	Image	Rôle
`reverse-proxy`	`reverse-proxy`	`nginx:alpine`	Point d'entrée, ports 80 et 443 publiés
`mon-application-1`	`app1`	`hashicorp/http-echo`	Application de test, répond « Bienvenue sur App 1 ! »
`mon-application-2`	`app2`	`hashicorp/http-echo`	Application de test, répond « Bienvenue sur App 2 ! »

Les trois conteneurs sont sur le même réseau Docker `web-net` (driver `bridge`, nom réel `reverse-proxy_web-net`). Seul Nginx publie des ports sur l'hôte : les applications ne sont joignables que via le proxy, sur leur port interne `3000`.

Arborescence

```
/root/reverse-proxy/
├── docker-compose.yml
└── nginx/
    └── conf.d/
        └── default.conf
```

Le dossier `nginx/conf.d` est monté en lecture seule dans le conteneur Nginx (`/etc/nginx/conf.d`).

Fichier docker-compose.yml

```
services:
  reverse-proxy:
    image: nginx:alpine
    container_name: reverse-proxy
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
    networks:
      - web-net

  mon-application-1:
    image: hashicorp/http-echo
    command: ["-listen=:3000", "-text=Bienvenue sur App 1 !"]
    container_name: app1
    restart: unless-stopped
    networks:
      - web-net

  mon-application-2:
    image: hashicorp/http-echo
    command: ["-listen=:3000", "-text=Bienvenue sur App 2 !"]
    container_name: app2
    restart: unless-stopped
    networks:
      - web-net

networks:
  web-net:
    driver: bridge
```

Points à retenir :

- `restart: unless-stopped` relance les conteneurs après un redémarrage du serveur.
- L'option `:ro` sur le volume empêche le conteneur de modifier la configuration.
- L'image `http-echo` écoute par défaut sur le port `5678` ; l'option `-listen=:3000` le change en `3000`. Nginx doit donc cibler le port `3000`.

Configuration Nginx

Le contenu exact de `nginx/conf.d/default.conf` n'apparaît pas dans la session. Voici une configuration type compatible avec cette architecture (à adapter, par exemple avec vos vrais noms de domaine) :

```
server {
    listen 80;
    server_name app1.example.com;

    location / {
        proxy_pass http://app1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

server {
    listen 80;
    server_name app2.example.com;

    location / {
        proxy_pass http://app2:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Dans `proxy_pass`, on utilise le nom du conteneur (`app1`, `app2`) : Docker résout ce nom grâce au réseau `web-net`.

Démarrage et vérification

1. Valider le fichier Compose et afficher la configuration finale :

   ```
   docker compose config
   ```

2. Lancer ou mettre à jour la pile :

   ```
   docker compose up -d
   ```

3. Vérifier l'état des conteneurs :

```
docker compose ps
```

4. Tester le proxy depuis le serveur :

```
curl -i http://localhost
```

5. Après toute modification de la configuration Nginx, la recharger sans couper le service :

```
docker compose exec reverse-proxy nginx -s reload
```

Dépannage

Erreur YAML : clé `services` définie deux fois

Message rencontré :

```
yaml: construct errors: line 3: mapping key "services" already defined at
line 1
```

Cause : le fichier `docker-compose.yml` contenait deux blocs `services:` au premier niveau, probablement après un copier-coller par-dessus l'ancien contenu. Solution : ne garder qu'un seul bloc `services:` puis relancer `docker compose config` pour valider avant tout `docker compose up -d`.

502 Bad Gateway au premier test

Lors du premier `curl -i http://localhost`, Nginx a renvoyé une erreur `502 Bad Gateway` : il a bien reçu la requête mais n'a pas pu joindre l'application derrière. À ce moment, les conteneurs `app1` et `app2` venaient d'être démarrés (quelques secondes) et le conteneur Nginx tournait depuis 3 heures. Les logs d'`app1` confirment que l'application écoute bien sur `:3000`.

Causes les plus fréquentes dans cette situation, à vérifier dans l'ordre :

1. **Mauvais port dans `proxy_pass`** : il doit pointer sur `3000` (et non `5678`, le port par défaut de l'image).
2. **Nom d'hôte non résolu ou obsolète** : Nginx résout les noms de conteneurs au chargement de la configuration. Si les applications ont été (re)créées après, recharger Nginx : `docker compose exec reverse-proxy nginx -s reload`, ou le redémarrer : `docker compose restart reverse-proxy`.
3. **Application pas encore prête** : attendre quelques secondes après le démarrage et retester.

4. **Conteneurs sur des réseaux différents** : vérifier que les trois services déclarent bien `web-net`.

Pour diagnostiquer, consulter les logs de chaque service :

```
docker compose logs reverse-proxy
docker compose logs mon-application-1
```

L'état final de la session ne montre pas de nouveau test après le dernier `docker compose up -d` : refaire un `curl -i http://localhost` pour confirmer que l'erreur a disparu.

Commandes utiles

Commande	Effet
`docker compose up -d`	Crée et démarre les services en arrière-plan
`docker compose down`	Arrête et supprime les conteneurs et le réseau
`docker compose ps`	Liste les conteneurs et leur état
`docker compose logs <service>`	Affiche les logs d'un service
`docker compose config`	Valide le fichier et affiche la configuration résolue
`docker compose restart <service>`	Redémarre un service
`docker compose exec reverse-proxy nginx -t`	Teste la syntaxe de la configuration Nginx

Pistes d'amélioration

- Activer le HTTPS sur le port 443 (certificats Let's Encrypt avec Certbot, ou un proxy comme Traefik ou Caddy qui les gère automatiquement).
- Ajouter un `depends_on` sur le service Nginx pour que les applications démarrent avant lui.
- Remplacer les applications de test par vos vrais services.